iroh 상위 프로토콜 분석

개요

iroh는 P2P QUIC 연결을 기반으로 다양한 상위 프로토콜을 구축할 수 있는 강력한 추상화 시스템을 제공합니다. 이 문서는 iroh의 프로토콜 추상화 메커니즘과 그 위에 구축된 상위 프로토콜들의 동작 방식을 상세히 분석합니다.

목차

1. 프로토콜 추상화 시스템
2. ALPN 기반 프로토콜 라우팅
3. 커스텀 프로토콜 개발 가이드
4. iroh-blobs 프로토콜
5. iroh-gossip 프로토콜
6. iroh-docs 프로토콜

프로토콜 추상화 시스템

ProtocolHandler 인터페이스

iroh의 프로토콜 추상화 시스템의 핵심은 `ProtocolHandler` 트레이트입니다. 이 트레이트는 들어오는 연결을 처리하는 표준화된 인터페이스를 제공합니다.

```
/// Handler for incoming connections.

///
```

```
/// A router accepts connections for arbitrary ALPN protocols.
pub trait ProtocolHandler: Send + Sync + std::fmt::Debug + 'static {
/// Optional interception point to handle the `Connecting` state.
fn on_connecting(
&self,
connecting: Connecting,
) -> impl Future<Output = Result<Connection, AcceptError>> + Send {
async move {
let conn = connecting.await?;
Ok(conn)
}
}

/// Handle an incoming connection.
fn accept(
&self,
connection: Connection,
) -> impl Future<Output = Result<(), AcceptError>> + Send;

/// Called when the router shuts down.
fn shutdown(&self) -> impl Future<Output = ()> + Send {
async move {}
}
```

```
}
```

주요 메서드 분석

1. **`on_connecting`** : 연결 설정 단계에서 호출되는 선택적 메서드
- 0-RTT 데이터 수신 등 고급 기능 구현 가능
- 기본 구현은 단순히 연결 완료를 대기
2. **`accept`** : 실제 연결 처리 로직을 구현하는 핵심 메서드
- 새로운 tokio 태스크에서 실행되어 장시간 실행 가능
- 연결별로 독립적인 처리 로직 구현
3. **`shutdown`** : 라우터 종료 시 호출되는 정리 메서드
- 리소스 정리 및 graceful shutdown 구현

Router 시스템

`Router` 는 여러 프로토콜을 관리하고 들어오는 연결을 적절한 핸들러로 라우팅하는 시스템입니다.

```
/// The built router.
#[derive(Clone, Debug)]
pub struct Router {
endpoint: Endpoint,
task: Arc<Mutex<Option<AbortOnDropHandle<()>>>>,
cancel_token: CancellationToken,
}
```

Router 생성 과정

```rust
let  router = Router::builder(endpoint)
.accept(b"/my/protocol/1", MyProtocolHandler)
.accept(b"/another/protocol/2", AnotherHandler)
.spawn();
```

내부 동작 메커니즘

1. **프로토콜 등록**: `RouterBuilder::accept` 를 통해 ALPN과 핸들러 매핑
2. **ALPN 설정**: 등록된 모든 ALPN을 endpoint에 설정
3. **Accept 루프**: 들어오는 연결을 지속적으로 수신
4. **핸들러 디스패치**: ALPN에 따라 적절한 핸들러로 연결 전달

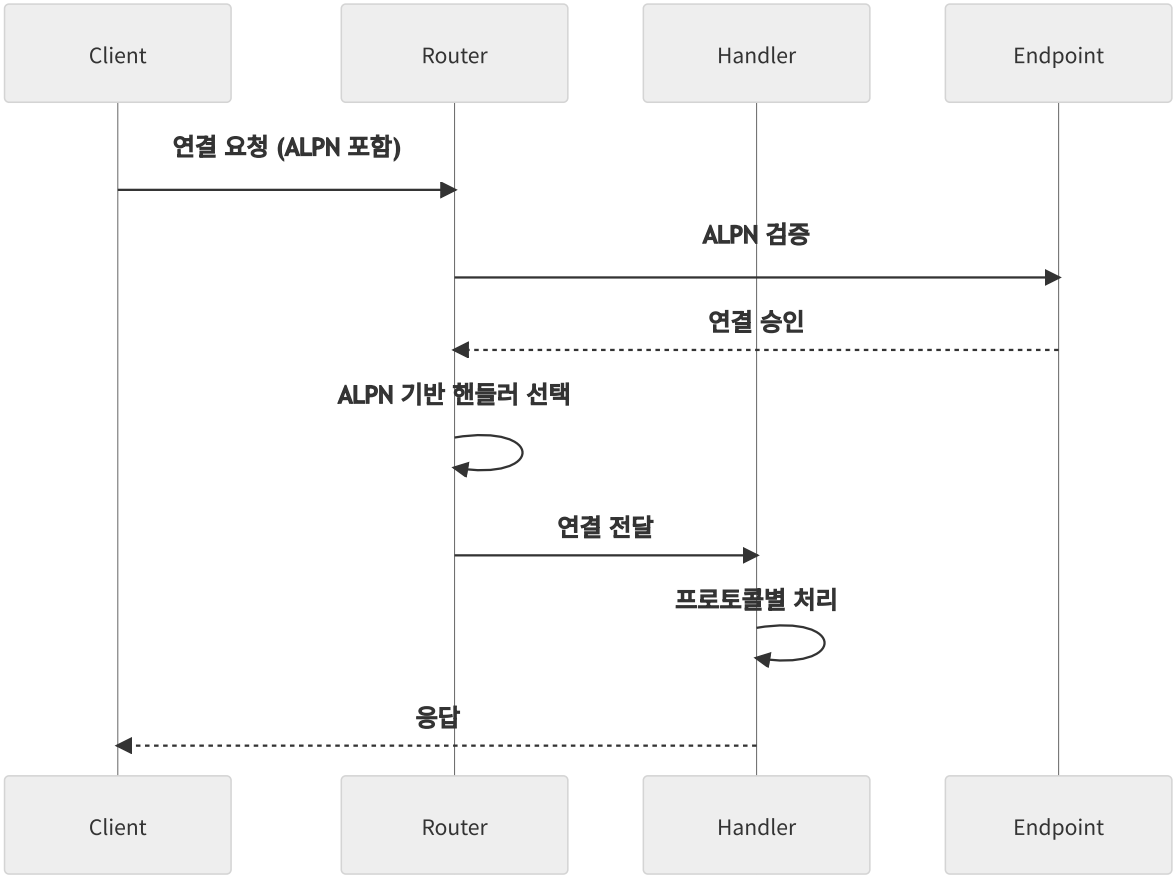
ALPN 기반 프로토콜 라우팅

ALPN (Application-Layer Protocol Negotiation)

ALPN은 TLS 핸드셰이크 과정에서 애플리케이션 계층 프로토콜을 협상하는 메커니즘입니다. iroh는 이를 활용하여 단일 endpoint에서 여러 프로토콜을 지원합니다.

```rust
/// Each protocol is identified by its ALPN string.
const ECHO_ALPN: &[u8] = b"iroh-example/echo/0";
const TRANSFER_ALPN: &[u8] = b"n0/iroh/transfer/example/0";
```

프로토콜 라우팅 과정



연결 처리 흐름

```
async fn handle_connection(incoming: Incoming, protocols: Arc<ProtocolMap>) {
let mut connecting = match incoming.accept() {
Ok(conn) => conn,
Err(err) => {
warn!("Ignoring connection: accepting failed: {err:#}");
return;
}
```

```
};

// ALPN 추출

let  alpn = match  connecting.alpn().await {

Ok(alpn) => alpn,

Err(err) => {

warn!("Ignoring connection: invalid handshake: {err:#}");

return;

}

};

// 핸들러 선택

let Some(handler) = protocols.get(&alpn) else {

warn!("Ignoring connection: unsupported ALPN protocol");

return;

};

// 연결 처리

match  handler.on_connecting(connecting).await {

Ok(connection) => {

if  let Err(err) = handler.accept(connection).await {

warn!("Handling incoming connection ended with error: {err}");

}

}

Err(err) => {

warn!("Handling incoming connecting ended with error: {err}");
```

```
}

}

}
```

커스텀 프로토콜 개발 가이드

기본 프로토콜 구현

간단한 에코 프로토콜 구현 예제:

```
#[derive(Debug, Clone)]

struct Echo;


impl ProtocolHandler for Echo {

async fn accept(&self, connection: Connection) -> Result<(), AcceptError> {

let node_id = connection.remote_node_id()?;

println!("accepted connection from {node_id}");


// 양방향 스트림 수락

let (mut send, mut recv) = connection.accept_bi().await?;


// 받은 데이터를 그대로 에코

let bytes_sent = tokio::io::copy(&mut recv, &mut send).await?;

println!("Copied over {bytes_sent} byte(s)");
```

```
// 전송 완료 신호
send.finish()?;

// 연결 종료 대기
connection.closed().await;

Ok(())
}
}
```

고급 프로토콜 패턴

1. 상태 관리 프로토콜

```
#[derive(Debug)]
struct StatefulProtocol {
state: Arc<Mutex<ProtocolState>>,
}

impl ProtocolHandler for StatefulProtocol {
async fn accept(&self, connection: Connection) -> Result<(), AcceptError> {
let mut state = self.state.lock().await;
// 상태 기반 처리 로직
```

```
Ok(())
}
async fn shutdown(&self) {
// 상태 정리
let mut state = self.state.lock().await;
state.cleanup().await;
}
}
```

2. 멀티 스트림 프로토콜

```
impl ProtocolHandler for MultiStreamProtocol {
async fn accept(&self, connection: Connection) -> Result<(), AcceptError> {
let mut tasks = JoinSet::new();
// 여러 스트림 동시 처리
loop {
tokio::select! {
stream = connection.accept_bi() => {
let (send, recv) = stream?;
tasks.spawn(handle_stream(send, recv));
}
result = tasks.join_next() => {
if let Some(Err(e)) = result {
warn!("Stream handling failed: {e}");
```

```
}
}
}
}
}
}
```

프로토콜 설계 원칙

1. **비동기 처리**: 모든 I/O 작업은 비동기로 구현
2. **에러 처리**: 적절한 에러 처리 및 복구 메커니즘
3. **리소스 관리**: 연결 및 스트림 리소스의 적절한 정리
4. **확장성**: 다중 연결 및 스트림 처리 고려
5. **보안**: 인증 및 권한 검사 구현

iroh-blobs 프로토콜

> **참고**: iroh-blobs 프로토콜은 현재 이 저장소에 구현되어 있지 않습니다. 다음은 iroh의 프로토콜 추상화 시스템을 바탕으로 한 설계 분석입니다.

BLAKE3 기반 콘텐츠 주소 지정 메커니즘

iroh-blobs는 BLAKE3 해시 함수를 사용하여 콘텐츠 주소 지정 시스템을 구현합니다. 이는 데이터 무결성과 중복 제거를 동시에 제공하는 핵심 메커니즘입니다.

해시 기반 식별자 시스템

```rust
// iroh-blobs 프로토콜 예상 구조
use blake3::{Hash as Blake3Hash, Hasher};

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub struct BlobHash(Blake3Hash);

impl BlobHash {
/// 데이터로부터 BLAKE3 해시 생성
pub fn from_bytes(data: &[u8]) -> Self {
let hash = blake3::hash(data);
Self(hash)
}
/// 스트리밍 방식으로 해시 계산
pub fn from_reader<R: std::io::Read>(mut reader: R) -> std::io::Result<Self>
let mut hasher = Hasher::new();
std::io::copy(&mut reader, &mut hasher)?;
Ok(Self(hasher.finalize()))
}
/// 데이터 무결성 검증
pub fn verify(&self, data: &[u8]) -> bool {
let computed = Self::from_bytes(data);
computed == *self
```

```rust
}

/// 해시를 바이트 배열로 변환

pub fn as_bytes(&self) -> &[u8; 32] {

self.0.as_bytes()

}

}



/// Blob 메타데이터

#[derive(Debug, Clone)]

pub struct BlobInfo {

pub hash: BlobHash,

pub size: u64,

pub chunk_count: usize,

pub created_at: std::time::SystemTime,

}
```

콘텐츠 주소 지정의 장점

1. **무결성 보장**: 해시가 곧 데이터의 지문 역할
2. **자동 중복 제거**: 동일한 데이터는 동일한 해시를 가짐
3. **분산 저장**: 해시를 통한 분산 노드 간 데이터 위치 결정
4. **캐싱 최적화**: 해시 기반 캐시 키 사용

Blob 전송 최적화 기법 및 청킹 전략

적응적 청킹 알고리즘

```rust
/// 청킹 전략 설정
#[derive(Debug, Clone)]
pub struct ChunkingConfig {
pub min_chunk_size: usize, // 최소 청크 크기 (예: 64KB)
pub max_chunk_size: usize, // 최대 청크 크기 (예: 16MB)
pub target_chunk_size: usize, // 목표 청크 크기 (예: 1MB)
pub rolling_hash_window: usize, // 롤링 해시 윈도우 크기
}

impl Default for ChunkingConfig {
fn default() -> Self {
Self {
min_chunk_size: 64 * 1024, // 64KB
max_chunk_size: 16 * 1024 * 1024, // 16MB
target_chunk_size: 1024 * 1024, // 1MB
rolling_hash_window: 64,
}
}
}

/// 청킹 결과
#[derive(Debug)]
```

```rust
pub struct Chunk {
pub hash: BlobHash,
pub offset: u64,
pub size: usize,
pub data: bytes::Bytes,
}

/// 콘텐츠 정의 청킹 (Content-Defined Chunking) 구현
pub struct ContentDefinedChunker {
config: ChunkingConfig,
}

impl ContentDefinedChunker {
pub fn new(config: ChunkingConfig) -> Self {
Self { config }
}
/// 데이터를 청크로 분할
pub fn chunk_data(&self, data: &[u8]) -> Vec<Chunk> {
let mut chunks = Vec::new();
let mut offset = 0;
while offset < data.len() {
let chunk_end = self.find_chunk_boundary(
&data[offset..],
```

```
self.config.target_chunk_size
);
let  chunk_size = std::cmp::min(
chunk_end,
std::cmp::max(
self.config.min_chunk_size,
std::cmp::min(
self.config.max_chunk_size,
data.len() - offset
)
)
);
let  chunk_data = &data[offset..offset + chunk_size];
let  chunk = Chunk {
hash: BlobHash::from_bytes(chunk_data),
offset: offset  as u64,
size: chunk_size,
data: bytes::Bytes::copy_from_slice(chunk_data),
};
chunks.push(chunk);
offset += chunk_size;
}
chunks
```

```
}

/// 롤링 해시를 사용한 청크 경계 찾기

fn find_chunk_boundary(&self, data: &[u8], target_size: usize) -> usize {

if data.len() <= target_size {

return data.len();

}

// Rabin fingerprinting 또는 유사한 롤링 해시 알고리즘 사용

// 여기서는 단순화된 구현

let mut hash = 0u32;

let window_size = std::cmp::min(self.config.rolling_hash_window, data.len());

for i in 0..window_size {

hash = hash.wrapping_mul(31).wrapping_add(data[i] as u32);

}

for i in window_size..std::cmp::min(target_size * 2, data.len()) {

if hash % 1024 == 0 && i >= self.config.min_chunk_size {

return i;

}

// 롤링 해시 업데이트

hash = hash.wrapping_sub(

(data[i - window_size] as u32).wrapping_mul(31_u32.pow(window_size as u32 - 1))

);

hash = hash.wrapping_mul(31).wrapping_add(data[i] as u32);

}
```

```
target_size
}
}
```

병렬 전송 및 최적화

```
/// Blob 전송 관리자
pub struct BlobTransferManager {
endpoint: iroh::Endpoint,
chunk_cache: Arc<RwLock<HashMap<BlobHash, bytes::Bytes>>>,
transfer_config: TransferConfig,
}

#[derive(Debug, Clone)]
pub struct TransferConfig {
pub max_concurrent_chunks: usize,
pub chunk_timeout: Duration,
pub retry_attempts: usize,
pub compression_enabled: bool,
}

impl BlobTransferManager {
/// 병렬 청크 전송
pub async fn send_blob(
```

```
&self,
connection: &iroh::endpoint::Connection,
blob_hash: BlobHash,
chunks: Vec<Chunk>,
) -> Result<(), BlobTransferError> {
let semaphore = Arc::new(Semaphore::new(self.transfer_config.max_concurrent_chu
let mut tasks = JoinSet::new();
// 청크별 전송 태스크 생성
for chunk in chunks {
let sem = semaphore.clone();
let conn = connection.clone();
let config = self.transfer_config.clone();
tasks.spawn(async move {
let _permit = sem.acquire().await?;
Self::send_chunk(&conn, chunk, &config).await
});
}
// 모든 전송 완료 대기
while let Some(result) = tasks.join_next().await {
result??; // JoinError와 BlobTransferError 모두 처리
}
Ok(())
}
```

```
/// 개별 청크 전송
async fn send_chunk(
connection: &iroh::endpoint::Connection,
chunk: Chunk,
config: &TransferConfig,
) -> Result<(), BlobTransferError> {
let mut attempts = 0;
while attempts < config.retry_attempts {
match Self::try_send_chunk(connection, &chunk, config).await {
Ok(()) => return Ok(()),
Err(e) if attempts + 1 >= config.retry_attempts => return Err(e),
Err(_) => {
attempts += 1;
tokio::time::sleep(Duration::from_millis(100 * attempts as u64)).await;
}
}
}
unreachable!()
}
async fn try_send_chunk(
connection: &iroh::endpoint::Connection,
chunk: &Chunk,
config: &TransferConfig,
```

```
) -> Result<(), BlobTransferError> {
let (mut  send, _recv) = connection.open_bi().await?;
// 청크 헤더 전송
let  header = ChunkHeader {
hash: chunk.hash,
offset: chunk.offset,
size: chunk.size,
compressed: config.compression_enabled,
};
let  header_bytes = bincode::serialize(&header)?;
send.write_all(&(header_bytes.len() as u32).to_le_bytes()).await?;
send.write_all(&header_bytes).await?;
// 청크 데이터 전송 (압축 옵션 적용)
let  data = if  config.compression_enabled {
compress_data(&chunk.data)?
} else {
chunk.data.clone()
};
send.write_all(&data).await?;
send.finish()?;
Ok(())
}
}
```

```rust
#[derive(Debug, Serialize, Deserialize)]
struct ChunkHeader {
hash: BlobHash,
offset: u64,
size: usize,
compressed: bool,
}
```

대용량 파일 처리 및 스트리밍 구현

스트리밍 기반 Blob 처리

```rust
/// 스트리밍 Blob 인터페이스
pub struct BlobStream {
chunks: Vec<ChunkInfo>,
total_size: u64,
current_chunk: usize,
chunk_fetcher: Arc<dyn ChunkFetcher>,
}

#[derive(Debug, Clone)]
pub struct ChunkInfo {
pub hash: BlobHash,
```

```
pub offset: u64,
pub size: usize,
}

#[async_trait::async_trait]
pub trait ChunkFetcher: Send + Sync {
async fn fetch_chunk(&self, hash: BlobHash) -> Result<bytes::Bytes, ChunkFetch
}

impl BlobStream {
pub fn new(chunks: Vec<ChunkInfo>, fetcher: Arc<dyn ChunkFetcher>) -> Self {
let total_size = chunks.iter().map(|c| c.size as u64).sum();
Self {
chunks,
total_size,
current_chunk: 0,
chunk_fetcher: fetcher,
}
}
/// AsyncRead 구현을 위한 스트리밍
pub async fn stream_to<W>(&mut self, mut writer: W) -> Result<u64, BlobStre
where
W: tokio::io::AsyncWrite + Unpin,
```

```
{
let mut bytes_written = 0u64;
for chunk_info in &self.chunks {
let chunk_data = self.chunk_fetcher
.fetch_chunk(chunk_info.hash)
.await?;
writer.write_all(&chunk_data).await?;
bytes_written += chunk_data.len() as u64;
}
writer.flush().await?;
Ok(bytes_written)
}
/// 특정 범위만 스트리밍 (HTTP Range 요청 지원)
pub async fn stream_range<W>(
&mut self,
mut writer: W,
start: u64,
end: u64,
) -> Result<u64, BlobStreamError>
where
W: tokio::io::AsyncWrite + Unpin,
{
let mut bytes_written = 0u64;
```

```
let mut current_offset = 0u64;
for chunk_info in &self.chunks {
let chunk_start = current_offset;
let chunk_end = current_offset + chunk_info.size as u64;
// 요청 범위와 겹치는 청크만 처리
if chunk_end > start && chunk_start < end {
let chunk_data = self.chunk_fetcher
.fetch_chunk(chunk_info.hash)
.await?;
let slice_start = if chunk_start < start {
(start - chunk_start) as usize
} else {
0
};
let slice_end = if chunk_end > end {
chunk_data.len() - (chunk_end - end) as usize
} else {
chunk_data.len()
};
let slice = &chunk_data[slice_start..slice_end];
writer.write_all(slice).await?;
bytes_written += slice.len() as u64;
}
```

```
current_offset = chunk_end;
if current_offset >= end {
break;
}
}
writer.flush().await?;
Ok(bytes_written)
}
}

/// 네트워크 기반 청크 페처 구현
pub struct NetworkChunkFetcher {
endpoint: iroh::Endpoint,
peer_cache: Arc<RwLock<HashMap<BlobHash, NodeId>>>,
}

#[async_trait::async_trait]
impl ChunkFetcher for NetworkChunkFetcher {
async fn fetch_chunk(&self, hash: BlobHash) -> Result<bytes::Bytes, ChunkFetch
// 1. 로컬 캐시 확인
if let Some(data) = self.check_local_cache(hash).await? {
return Ok(data);
}
```

```
// 2. 피어에서 청크 요청
let peer_id = self.find_chunk_provider(hash).await?;
let data = self.request_chunk_from_peer(peer_id, hash).await?;
// 3. 로컬 캐시에 저장
self.cache_chunk(hash, &data).await?;
Ok(data)
}
}
```

메모리 효율적인 대용량 파일 처리

```
/// 메모리 효율적인 Blob 프로세서
pub struct LargeBlobProcessor {
chunk_size: usize,
temp_dir: PathBuf,
max_memory_usage: usize,
}

impl LargeBlobProcessor {
/// 대용량 파일을 청크 단위로 처리
pub async fn process_large_file<P, F>(
&self,
file_path: P,
```

```
mut processor: F,
) -> Result<BlobInfo, ProcessingError>
where
P: AsRef<Path>,
F: FnMut(&[u8]) -> Result<(), ProcessingError>,
{
let mut file = tokio::fs::File::open(file_path).await?;
let mut buffer = vec![0u8; self.chunk_size];
let mut hasher = blake3::Hasher::new();
let mut total_size = 0u64;
let mut chunk_count = 0;
loop {
let bytes_read = file.read(&mut buffer).await?;
if bytes_read == 0 {
break;
}
let chunk = &buffer[..bytes_read];
hasher.update(chunk);
processor(chunk)?;
total_size += bytes_read as u64;
chunk_count += 1;
// 메모리 사용량 모니터링
if chunk_count % 100 == 0 {
```

```
self.check_memory_usage().await?;
}
}
Ok(BlobInfo {
hash: BlobHash(hasher.finalize()),
size: total_size,
chunk_count,
created_at: std::time::SystemTime::now(),
})
}
async fn check_memory_usage(&self) -> Result<(), ProcessingError> {
// 메모리 사용량 확인 및 필요시 가비지 컬렉션
// 시스템별 구현 필요
Ok(())
}
}
```

iroh-gossip 프로토콜

> **참고**: iroh-gossip 프로토콜은 현재 이 저장소에 구현되어 있지 않습니다. 다음은 iroh의 프로토콜 추상화 시스템을 바탕으로 한 설계 분석입니다.

Pub-Sub 오버레이 네트워크 구축 알고리즘

iroh-gossip은 분산 pub-sub 시스템을 구현하여 효율적인 메시지 전파를 제공합니다. 이는 epidemic 알고리즘을 기반으로 한 확장 가능한 오버레이 네트워크를 구축합니다.

토픽 기반 구독 시스템

```
use blake3::Hash as Blake3Hash;

use std::collections::{HashMap, HashSet};

use tokio::sync::{broadcast, RwLock};


/// Gossip 토픽 식별자

#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]

pub struct GossipTopic(Blake3Hash);


impl GossipTopic {

pub fn new(name: &str) -> Self {

Self(blake3::hash(name.as_bytes()))

}

pub fn from_bytes(data: &[u8]) -> Self {

Self(blake3::hash(data))

}

}


/// Gossip 메시지

#[derive(Debug, Clone)]

pub struct GossipMessage {
```

```
pub id: MessageId,
pub topic: GossipTopic,
pub payload: bytes::Bytes,
pub timestamp: std::time::SystemTime,
pub author: iroh_base::NodeId,
pub signature: Option<Signature>,
pub ttl: u32,
}

/// 메시지 고유 식별자
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
pub struct MessageId(Blake3Hash);

impl MessageId {
pub fn new(topic: GossipTopic, payload: &[u8], author: iroh_base::NodeId, time
let mut hasher = blake3::Hasher::new();
hasher.update(topic.0.as_bytes());
hasher.update(payload);
hasher.update(author.as_bytes());
hasher.update(&timestamp.duration_since(std::time::UNIX_EPOCH).unwrap().as_secs(
Self(hasher.finalize())
}
}
```

```rust
/// Gossip 노드 구현
pub struct GossipNode {
endpoint: iroh::Endpoint,
subscriptions: Arc<RwLock<HashMap<GossipTopic, broadcast::Sender<GossipMessage>>
peers: Arc<RwLock<HashMap<iroh_base::NodeId, PeerConnection>>>,
message_cache: Arc<RwLock<MessageCache>>,
topology_manager: Arc<TopologyManager>,
config: GossipConfig,
}

#[derive(Debug, Clone)]
pub struct GossipConfig {
pub max_message_size: usize,
pub message_cache_size: usize,
pub message_ttl: u32,
pub fanout: usize, // 메시지 전파 시 선택할 피어 수
pub heartbeat_interval: Duration,
pub prune_interval: Duration,
}

impl Default for GossipConfig {
fn default() -> Self {
```

```
Self {
max_message_size: 1024 * 1024, // 1MB
message_cache_size: 10000,
message_ttl: 30,
fanout: 6,
heartbeat_interval: Duration::from_secs(1),
prune_interval: Duration::from_secs(60),
}
}
}

impl GossipNode {
pub fn new(endpoint: iroh::Endpoint, config: GossipConfig) -> Self {
Self {
endpoint,
subscriptions: Arc::new(RwLock::new(HashMap::new())),
peers: Arc::new(RwLock::new(HashMap::new())),
message_cache: Arc::new(RwLock::new(MessageCache::new(config.message_cache_size)
topology_manager: Arc::new(TopologyManager::new()),
config,
}
}
/// 토픽 구독
```

```
pub  async  fn  subscribe(&self, topic: GossipTopic) -> broadcast::Receiver<Goss

let  mut  subscriptions = self.subscriptions.write().await;

if  let Some(sender) = subscriptions.get(&topic) {

sender.subscribe()

} else {

let (sender, receiver) = broadcast::channel(1000);

subscriptions.insert(topic, sender);

// 피어들에게 구독 정보 전파

self.announce_subscription(topic).await;

receiver

}

}

/// 메시지 발행

pub  async  fn  publish(&self, topic: GossipTopic, payload: bytes::Bytes) -> Res

let  message = GossipMessage {

id: MessageId::new(topic, &payload, self.endpoint.node_id(), std::time::SystemTi

topic,

payload,

timestamp: std::time::SystemTime::now(),

author: self.endpoint.node_id(),

signature: None, // TODO: 서명 구현

ttl: self.config.message_ttl,

};
```

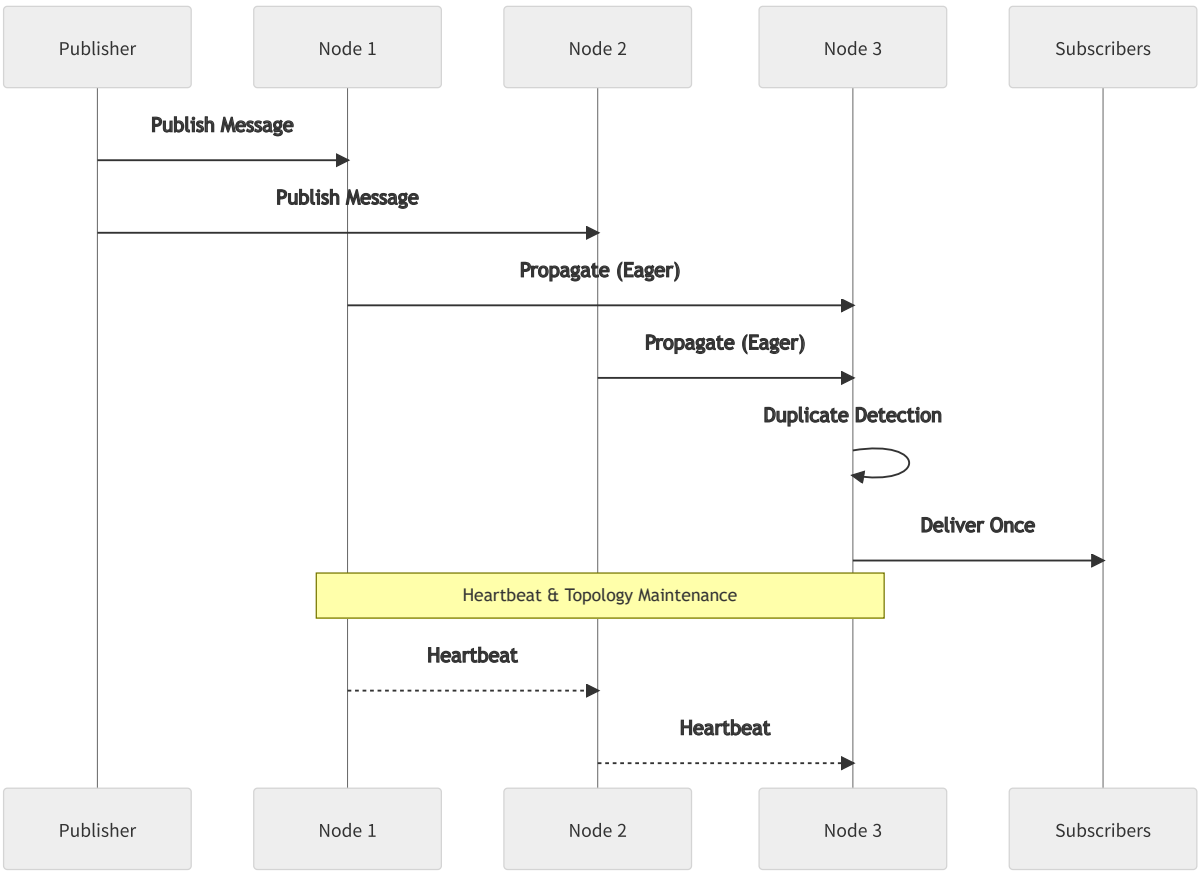
```
// 로컬 구독자들에게 전달
self.deliver_local_message(&message).await?;
// 피어들에게 전파
self.propagate_message(message).await?;
Ok(())
}
async fn announce_subscription(&self, topic: GossipTopic) {
let peers = self.peers.read().await;
for peer in peers.values() {
let _ = peer.send_subscription_announcement(topic).await;
}
}
async fn deliver_local_message(&self, message: &GossipMessage) -> Result<(), G
let subscriptions = self.subscriptions.read().await;
if let Some(sender) = subscriptions.get(&message.topic) {
let _ = sender.send(message.clone());
}
Ok(())
}
async fn propagate_message(&self, message: GossipMessage) -> Result<(), Gossip
// 메시지 캐시에 저장
{
let mut cache = self.message_cache.write().await;
```

```
if cache.contains(&message.id) {
return Ok(()); // 이미 처리된 메시지
}
cache.insert(message.id, message.clone());
}
// 전파할 피어 선택
let selected_peers = self.topology_manager
.select_propagation_peers(&message.topic, self.config.fanout)
.await;
// 병렬 전송
let mut tasks = tokio::task::JoinSet::new();
for peer_id in selected_peers {
let peers = self.peers.read().await;
if let Some(peer) = peers.get(&peer_id) {
let peer = peer.clone();
let msg = message.clone();
tasks.spawn(async move {
peer.send_message(msg).await
});
}
}
// 전송 결과 수집
while let Some(result) = tasks.join_next().await {
```

```
if  let Err(e) = result {
tracing::warn!("Failed to propagate message: {e}");
}
}
Ok(())
}
}
```

메시지 전파 및 중복 제거 메커니즘

Epidemic 알고리즘



중복 제거 전략

1. **메시지 ID 기반 중복 검사**: BLAKE3 해시를 사용한 고유 식별자
2. **Bloom Filter**: 메모리 효율적인 중복 검사
3. **TTL 관리**: 메시지 생존 시간 제한으로 메모리 사용량 제어
4. **백프레셔 제어**: 네트워크 혼잡 시 전송 속도 조절

확장성 및 내결함성 보장 방법

동적 토폴로지 관리

토폴로지 관리자는 네트워크의 효율성과 내결함성을 보장하기 위해 피어 관계를 동적으로 관리합니다.

```
/// 토폴로지 관리자
pub struct TopologyManager {
target_degree: usize,
max_degree: usize,
peer_scores: Arc<RwLock<HashMap<iroh_base::NodeId, PeerScore>>>,
topic_meshes: Arc<RwLock<HashMap<GossipTopic, TopicMesh>>>,
}

#[derive(Debug, Clone)]
pub struct PeerScore {
pub reliability: f64, // 메시지 전달 신뢰도
pub latency: Duration, // 평균 지연시간
pub bandwidth: u64, // 대역폭 추정치
pub uptime: Duration, // 연결 지속 시간
pub last_seen: std::time::Instant,
}

#[derive(Debug)]
pub struct TopicMesh {
pub eager_peers: HashSet<iroh_base::NodeId>, // 적극적 전파 피어
```

```rust
pub lazy_peers: HashSet<iroh_base::NodeId>, // 지연 전파 피어
pub last_prune: std::time::Instant,
}


impl TopologyManager {
/// 메시지 전파를 위한 최적 피어 선택
pub async fn select_propagation_peers(
&self,
topic: &GossipTopic,
fanout: usize,
) -> Vec<iroh_base::NodeId> {
let meshes = self.topic_meshes.read().await;
let scores = self.peer_scores.read().await;
if let Some(mesh) = meshes.get(topic) {
let mut candidates: Vec<_> = mesh.eager_peers
.iter()
.filter_map(|peer_id| {
scores.get(peer_id).map(|score| (*peer_id, score.clone()))
})
.collect();
// 다중 기준 점수 계산 및 정렬
candidates.sort_by(|a, b| {
let score_a = self.calculate_propagation_score(&a.1);
```

```
let score_b = self.calculate_propagation_score(&b.1);
score_b.partial_cmp(&score_a).unwrap_or(std::cmp::Ordering::Equal)
});
candidates.into_iter()
.take(fanout)
.map(|(peer_id, _)| peer_id)
.collect()
} else {
Vec::new()
}
}
fn calculate_propagation_score(&self, peer_score: &PeerScore) -> f64 {
// 가중 평균을 사용한 종합 점수 계산
let reliability_weight = 0.4;
let latency_weight = 0.3;
let bandwidth_weight = 0.2;
let uptime_weight = 0.1;
let latency_score = 1.0 / (1.0 + peer_score.latency.as_millis() as f64 / 1000.0
let bandwidth_score = (peer_score.bandwidth as f64).log10() / 10.0;
let uptime_score = (peer_score.uptime.as_secs() as f64).log10() / 10.0;
reliability_weight * peer_score.reliability +
latency_weight * latency_score +
bandwidth_weight * bandwidth_score +
```

```
uptime_weight * uptime_score
}
}
```

내결함성 메커니즘

```
/// 장애 감지 및 복구 시스템
pub struct FailureDetector {
heartbeat_timeout: Duration,
max_missed_heartbeats: usize,
peer_states: HashMap<iroh_base::NodeId, PeerState>,
}

#[derive(Debug)]
struct PeerState {
last_heartbeat: std::time::Instant,
missed_heartbeats: usize,
is_suspected: bool,
}

impl FailureDetector {
pub fn check_failures(&mut self) -> Vec<iroh_base::NodeId> {
let now = std::time::Instant::now();
```

```
let mut failed_peers = Vec::new();
for (peer_id, state) in &mut self.peer_states {
if now.duration_since(state.last_heartbeat) > self.heartbeat_timeout {
state.missed_heartbeats += 1;
if state.missed_heartbeats >= self.max_missed_heartbeats {
failed_peers.push(*peer_id);
} else {
state.is_suspected = true;
}
}
}
// 실패한 피어 제거 및 토폴로지 재구성
for peer_id in &failed_peers {
self.peer_states.remove(peer_id);
}
failed_peers
}
}
```

iroh-docs 프로토콜

> **참고**: iroh-docs 프로토콜은 현재 이 저장소에 구현되어 있지 않습니다. 다음은 iroh의

Eventually-Consistent Key-Value 저장소 구현

iroh-docs는 분산 환경에서 최종 일관성(eventual consistency)을 보장하는 키-값 저장소를 구

데이터 모델 및 구조

```rust
use blake3::Hash as Blake3Hash;
use std::collections::BTreeMap;
use serde::{Serialize, Deserialize};

/// 문서 고유 식별자
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
pub struct DocumentId(Blake3Hash);

impl DocumentId {
pub fn new(namespace: &str, name: &str) -> Self {
let mut hasher = blake3::Hasher::new();
hasher.update(namespace.as_bytes());
hasher.update(b"/");
hasher.update(name.as_bytes());
Self(hasher.finalize())
```

```
}
pub fn random() -> Self {
Self(blake3::hash(&rand::random::<[u8; 32]>()))
}
}


/// 키 타입 (계층적 구조 지원)
#[derive(Debug, Clone, PartialEq, Eq, PartialOrd, Ord, Serialize, Deserialize)]
pub struct Key(Vec<String>);


impl Key {
pub fn new(path: &[&str]) -> Self {
Self(path.iter().map(|s| s.to_string()).collect())
}
pub fn from_string(path: &str) -> Self {
Self(path.split('/').map(|s| s.to_string()).collect())
}
pub fn parent(&self) -> Option<Key> {
if self.0.len() > 1 {
Some(Key(self.0[..self.0.len()-1].to_vec()))
} else {
None
}
```

```
}

pub fn child(&self, name: &str) -> Key {

let mut path = self.0.clone();

path.push(name.to_string());

Key(path)

}

}



/// 문서 엔트리

#[derive(Debug, Clone, Serialize, Deserialize)]

pub struct Entry {

pub value: bytes::Bytes,

pub timestamp: LogicalTimestamp,

pub author: iroh_base::NodeId,

pub signature: Option<Signature>,

pub content_hash: Blake3Hash,

pub metadata: EntryMetadata,

}



#[derive(Debug, Clone, Serialize, Deserialize)]

pub struct EntryMetadata {

pub content_type: Option<String>,

pub encoding: Option<String>,
```

```
pub tags: Vec<String>,

pub created_at: std::time::SystemTime,

pub modified_at: std::time::SystemTime,

}



/// 논리적 타임스탬프 (Lamport timestamp + 노드 ID)

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Serialize, Deserial

pub struct LogicalTimestamp {

pub counter: u64,

pub node_id: iroh_base::NodeId,

}



impl LogicalTimestamp {

pub fn new(counter: u64, node_id: iroh_base::NodeId) -> Self {

Self { counter, node_id }

}

pub fn increment(&mut self) {

self.counter += 1;

}

pub fn update(&mut self, other: LogicalTimestamp) {

self.counter = std::cmp::max(self.counter, other.counter) + 1;

}

}
```

```
/// 분산 문서
#[derive(Debug, Clone)]
pub struct Document {
pub id: DocumentId,
pub entries: BTreeMap<Key, Entry>,
pub vector_clock: VectorClock,
pub local_clock: u64,
pub node_id: iroh_base::NodeId,
pub access_control: AccessControl,
}

/// 벡터 클록 구현
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct VectorClock {
clocks: HashMap<iroh_base::NodeId, u64>,
}

impl VectorClock {
pub fn new() -> Self {
Self {
clocks: HashMap::new(),
}
```

```
}

pub fn increment(&mut self, node_id: iroh_base::NodeId) {

let counter = self.clocks.entry(node_id).or_insert(0);

*counter += 1;

}

pub fn update(&mut self, other: &VectorClock) {

for (node_id, &other_counter) in &other.clocks {

let counter = self.clocks.entry(*node_id).or_insert(0);

*counter = std::cmp::max(*counter, other_counter);

}

}

pub fn compare(&self, other: &VectorClock) -> ClockOrdering {

let mut less_than = false;

let mut greater_than = false;

let all_nodes: HashSet<_> = self.clocks.keys()

.chain(other.clocks.keys())

.collect();

for node_id in all_nodes {

let self_counter = self.clocks.get(node_id).unwrap_or(&0);

let other_counter = other.clocks.get(node_id).unwrap_or(&0);

match self_counter.cmp(other_counter) {

std::cmp::Ordering::Less => less_than = true,

std::cmp::Ordering::Greater => greater_than = true,
```

```
std::cmp::Ordering::Equal => {}

}

}

match (less_than, greater_than) {

(true, false) => ClockOrdering::Before,

(false, true) => ClockOrdering::After,

(false, false) => ClockOrdering::Equal,

(true, true) => ClockOrdering::Concurrent,

}

}

}
```

```
#[derive(Debug, PartialEq)]

pub enum ClockOrdering {

Before,

After,

Equal,

Concurrent,

}
```

충돌 해결 및 동기화 알고리즘

다중 전략 충돌 해결

```rust
/// 충돌 해결 전략
#[derive(Debug, Clone)]
pub enum ConflictResolution {
LastWriterWins,
FirstWriterWins,
Custom(Box<dyn ConflictResolver>),
Manual, // 사용자가 수동으로 해결
}

pub trait ConflictResolver: Send + Sync {
fn resolve(&self, local: &Entry, remote: &Entry) -> ConflictResolution;
}

impl Document {
/// 다른 문서와 병합
pub fn merge(&mut self, other: &Document) -> Result<MergeResult, DocumentErro
if self.id != other.id {
return Err(DocumentError::DocumentIdMismatch);
}
let mut conflicts = Vec::new();
let mut merged_entries = 0;
// 벡터 클록 업데이트
```

```
self.vector_clock.update(&other.vector_clock);

// 엔트리별 병합

for (key, other_entry) in &other.entries {

match  self.entries.get(key) {

Some(local_entry) => {

match  self.resolve_conflict(local_entry, other_entry) {

ConflictResolution::UseRemote => {

self.entries.insert(key.clone(), other_entry.clone());

merged_entries += 1;

}

ConflictResolution::UseLocal => {

// 로컬 엔트리 유지

}

ConflictResolution::Conflict(conflict) => {

conflicts.push(conflict);

}

}

}

None => {

// 새로운 엔트리 추가

self.entries.insert(key.clone(), other_entry.clone());

merged_entries += 1;

}
```

```
}
}
Ok(MergeResult {
merged_entries,
conflicts,
})
}
fn  resolve_conflict(&self, local: &Entry, remote: &Entry) -> ConflictResolution
// 타임스탬프 기반 해결
match  local.timestamp.cmp(&remote.timestamp) {
std::cmp::Ordering::Less => ConflictResolution::UseRemote,
std::cmp::Ordering::Greater => ConflictResolution::UseLocal,
std::cmp::Ordering::Equal => {
// 동일한 타임스탬프인 경우 노드 ID로 결정 (결정적 해결)
if  remote.author < local.author {
ConflictResolution::UseRemote
} else {
ConflictResolution::UseLocal
}
}
}
}
}
/// 키-값 쌍 설정
```

```
pub fn set(&mut self, key: Key, value: bytes::Bytes) -> Result<(), DocumentEr
// 접근 권한 확인
if !self.access_control.can_write(&key, &self.node_id) {
return Err(DocumentError::AccessDenied);
}
// 로컬 클록 증가
self.local_clock += 1;
self.vector_clock.increment(self.node_id);
let entry = Entry {
content_hash: blake3::hash(&value),
value,
timestamp: LogicalTimestamp::new(self.local_clock, self.node_id),
author: self.node_id,
signature: None, // TODO: 서명 구현
metadata: EntryMetadata {
content_type: None,
encoding: None,
tags: Vec::new(),
created_at: std::time::SystemTime::now(),
modified_at: std::time::SystemTime::now(),
},
};
self.entries.insert(key, entry);
```

```
Ok(())
}
/// 키로 값 조회
pub fn get(&self, key: &Key) -> Option<&Entry> {
if !self.access_control.can_read(key, &self.node_id) {
return None;
}
self.entries.get(key)
}
/// 키 삭제 (tombstone 방식)
pub fn delete(&mut self, key: &Key) -> Result<(), DocumentError> {
if !self.access_control.can_write(key, &self.node_id) {
return Err(DocumentError::AccessDenied);
}
// 삭제 마커 설정 (실제 삭제하지 않음)
self.set(key.clone(), bytes::Bytes::new())?; // 빈 값으로 설정
Ok(())
}
}


#[derive(Debug)]
pub struct MergeResult {
pub merged_entries: usize,
```

```
pub conflicts: Vec<Conflict>,
}

#[derive(Debug)]
pub struct Conflict {
pub key: Key,
pub local_entry: Entry,
pub remote_entry: Entry,
}
```

분산 환경에서의 일관성 보장 메커니즘

동기화 프로토콜

```
/// 문서 동기화 관리자
pub struct DocumentSyncManager {
endpoint: iroh::Endpoint,
documents: Arc<RwLock<HashMap<DocumentId, Document>>>,
sync_state: Arc<RwLock<HashMap<DocumentId, SyncState>>>,
config: SyncConfig,
}

#[derive(Debug, Clone)]
pub struct SyncConfig {
```

```
pub  sync_interval: Duration,

pub  batch_size: usize,

pub  max_concurrent_syncs: usize,

pub  compression_enabled: bool,

}


#[derive(Debug)]

struct SyncState {

last_sync: std::time::Instant,

peers: HashMap<iroh_base::NodeId, PeerSyncState>,

pending_changes: VecDeque<ChangeEvent>,

}


#[derive(Debug)]

struct PeerSyncState {

last_vector_clock: VectorClock,

last_sync_time: std::time::Instant,

sync_in_progress: bool,

}


#[derive(Debug, Clone)]

struct ChangeEvent {

document_id: DocumentId,
```

```
key: Key,
change_type: ChangeType,
timestamp: LogicalTimestamp,
}

#[derive(Debug, Clone)]
enum ChangeType {
Insert,
Update,
Delete,
}

impl DocumentSyncManager {
/// 피어와 문서 동기화
pub  async  fn  sync_with_peer(
&self,
peer_id: iroh_base::NodeId,
document_id: DocumentId,
) -> Result<SyncResult, SyncError> {
let  connection = self.endpoint.connect_by_node_id(peer_id, b"iroh-docs/sync/1")
// 1. 벡터 클록 교환
let  local_clock = {
let  documents = self.documents.read().await;
```

```
documents.get(&document_id)
.map(|doc| doc.vector_clock.clone())
.unwrap_or_else(VectorClock::new)
};
let (mut send, mut recv) = connection.open_bi().await?;
// 동기화 요청 전송
let sync_request = SyncRequest {
document_id,
vector_clock: local_clock.clone(),
request_type: SyncRequestType::Full,
};
let request_data = bincode::serialize(&sync_request)?;
send.write_all(&(request_data.len() as u32).to_le_bytes()).await?;
send.write_all(&request_data).await?;
// 응답 수신
let mut response_len_buf = [0u8; 4];
recv.read_exact(&mut response_len_buf).await?;
let response_len = u32::from_le_bytes(response_len_buf) as usize;
let mut response_buf = vec![0u8; response_len];
recv.read_exact(&mut response_buf).await?;
let sync_response: SyncResponse = bincode::deserialize(&response_buf)?;
// 2. 차이점 계산 및 적용
let mut merge_result = MergeResult {
```

```
merged_entries: 0,
conflicts: Vec::new(),
};
{
let mut documents = self.documents.write().await;
if let Some(local_doc) = documents.get_mut(&document_id) {
for remote_entry in sync_response.entries {
// 원격 엔트리 적용
match local_doc.entries.get(&remote_entry.key) {
Some(local_entry) => {
if remote_entry.entry.timestamp > local_entry.timestamp {
local_doc.entries.insert(remote_entry.key, remote_entry.entry);
merge_result.merged_entries += 1;
}
}
None => {
local_doc.entries.insert(remote_entry.key, remote_entry.entry);
merge_result.merged_entries += 1;
}
}
}
// 벡터 클록 업데이트
local_doc.vector_clock.update(&sync_response.vector_clock);
```

```
}

}

// 3. 로컬 변경사항 전송

let  local_changes = self.get_changes_since(&document_id, &sync_response.vector_

if !local_changes.is_empty() {

let  changes_data = bincode::serialize(&local_changes)?;

send.write_all(&(changes_data.len() as u32).to_le_bytes()).await?;

send.write_all(&changes_data).await?;

}

send.finish()?;

Ok(SyncResult {

peer_id,

document_id,

merged_entries: merge_result.merged_entries,

conflicts: merge_result.conflicts,

sync_duration: std::time::Instant::now().duration_since(std::time::Instant::now(

})

}

async  fn  get_changes_since(

&self,

document_id: &DocumentId,

since_clock: &VectorClock,

) -> Result<Vec<SyncEntry>, SyncError> {
```

```
let documents = self.documents.read().await;
let mut changes = Vec::new();
if let Some(document) = documents.get(document_id) {
for (key, entry) in &document.entries {
// 벡터 클록 비교를 통해 새로운 엔트리만 선택
if self.is_entry_newer(entry, since_clock) {
changes.push(SyncEntry {
key: key.clone(),
entry: entry.clone(),
});
}
}
}
Ok(changes)
}
fn is_entry_newer(&self, entry: &Entry, since_clock: &VectorClock) -> bool {
// 엔트리의 타임스탬프가 벡터 클록보다 새로운지 확인
let entry_counter = since_clock.clocks.get(&entry.author).unwrap_or(&0);
entry.timestamp.counter > *entry_counter
}
}

#[derive(Debug, Serialize, Deserialize)]
```

```rust
struct SyncRequest {
document_id: DocumentId,
vector_clock: VectorClock,
request_type: SyncRequestType,
}

#[derive(Debug, Serialize, Deserialize)]
enum SyncRequestType {
Full,
Incremental,
KeysOnly,
}

#[derive(Debug, Serialize, Deserialize)]
struct SyncResponse {
document_id: DocumentId,
vector_clock: VectorClock,
entries: Vec<SyncEntry>,
}

#[derive(Debug, Serialize, Deserialize)]
struct SyncEntry {
key: Key,
```

```
entry: Entry,
}

#[derive(Debug)]
pub struct SyncResult {
pub peer_id: iroh_base::NodeId,
pub document_id: DocumentId,
pub merged_entries: usize,
pub conflicts: Vec<Conflict>,
pub sync_duration: Duration,
}
```

접근 제어 및 보안

```
/// 문서 접근 제어
#[derive(Debug, Clone)]
pub struct AccessControl {
owner: iroh_base::NodeId,
permissions: HashMap<Key, KeyPermissions>,
default_permissions: KeyPermissions,
}

#[derive(Debug, Clone)]
```

```
pub struct KeyPermissions {
pub readers: HashSet<iroh_base::NodeId>,
pub writers: HashSet<iroh_base::NodeId>,
pub public_read: bool,
pub public_write: bool,
}


impl AccessControl {
pub fn new(owner: iroh_base::NodeId) -> Self {
Self {
owner,
permissions: HashMap::new(),
default_permissions: KeyPermissions {
readers: HashSet::new(),
writers: HashSet::new(),
public_read: false,
public_write: false,
},
}
}
pub fn can_read(&self, key: &Key, node_id: &iroh_base::NodeId) -> bool {
if *node_id == self.owner {
return true;
```

```
}
let permissions = self.permissions.get(key)
.unwrap_or(&self.default_permissions);
permissions.public_read || permissions.readers.contains(node_id)
}
pub fn can_write(&self, key: &Key, node_id: &iroh_base::NodeId) -> bool {
if *node_id == self.owner {
return true;
}
let permissions = self.permissions.get(key)
.unwrap_or(&self.default_permissions);
permissions.public_write || permissions.writers.contains(node_id)
}
pub fn grant_read(&mut self, key: Key, node_id: iroh_base::NodeId) {
let permissions = self.permissions.entry(key).or_insert_with(|| self.default_pe
permissions.readers.insert(node_id);
}
pub fn grant_write(&mut self, key: Key, node_id: iroh_base::NodeId) {
let permissions = self.permissions.entry(key).or_insert_with(|| self.default_pe
permissions.writers.insert(node_id);
}
}
```

결론

iroh의 프로토콜 추상화 시스템은 다음과 같은 장점을 제공합니다:

1. **유연성**: ALPN 기반 라우팅으로 다양한 프로토콜 지원
2. **확장성**: 새로운 프로토콜 쉽게 추가 가능
3. **성능**: 비동기 처리 및 최적화된 네트워킹
4. **안정성**: 강력한 에러 처리 및 복구 메커니즘

이러한 기반 위에 구축된 상위 프로토콜들(iroh-blobs, iroh-gossip, iroh-docs)은 각각 특화